

Universidad de Antioquia

Departamento de Ingeniería de Sistemas

Arquitectura de Software

Laboratorio 2

Entorno colaborativo de integración y despliegue continuo (CI/CD)

Spring Boot, GitHub Actions, SonarCloud, Docker Hub y Render

Estudiante

David Arango Pineda

C.C. 1000758519

Profesor

Diego Botia

Octubre de 2026

1. Objetivo y resumen

Este informe documenta el paso a paso seguido para completar el pipeline de integración y despliegue continuo de una aplicación Spring Boot (API REST con datos aleatorios generados con Faker). El pipeline compila el código, ejecuta las pruebas, mide la calidad y la cobertura con SonarCloud y JaCoCo, construye una imagen Docker, la publica en Docker Hub y la despliega automáticamente en **Render** cuando los cambios se fusionan en la rama principal. Se trabajó con la estrategia *trunk based*: ramas cortas que se integran a *main* mediante pull requests.

Enlaces del proyecto:

Recurso	Dirección
Repositorio (GitHub)	https://github.com/DavidAraangoPineda/lab2p2026
Pull request #1	https://github.com/DavidAraangoPineda/lab2p2026/pull/1
Imagen en Docker Hub	https://hub.docker.com/r/davidarango6/lab2p2026
Proyecto en SonarCloud	https://sonarcloud.io/project/overview?id=DavidAraangoPineda_lab2p2026
Aplicación en Render	https://lab2p2026-latest.onrender.com

2. Arquitectura del pipeline

El archivo `.github/workflows/build.yml` define cuatro trabajos encadenados con needs: si uno falla, los siguientes no se ejecutan.

Trabajo	Qué hace	Cuándo corre
tests (Unit tests)	Ejecuta <code>mvn -B test</code> con JUnit y JaCoCo.	Push a main / feature/deploy-render y PR a main
sonar (SonarCloud analysis)	Compila con <code>mvn clean verify</code> y envía el análisis estático y la cobertura a SonarCloud.	Igual que tests, después de tests
build (Build)	Empaqueta el JAR, lo sube como artefacto y publica la imagen Docker en Docker Hub.	Después de sonar
deploy (Deploy to Render)	Llama al <i>deploy hook</i> de Render para que descargue la imagen nueva y la ejecute.	Solo en push a main (no en PRs)

Disparadores configurados (se agregó la rama nueva, como pide el enunciado):

```
on:
  workflow_dispatch:
  push:
    branches:
      - main
      - feature/deploy-render
  pull_request:
    branches:
      - main
```

3. Paso a paso

3.1 Verificación local

Antes de tocar el pipeline se comprobó en local que la aplicación compila y que las 7 pruebas unitarias del controlador pasan con `./mvnw test`. Se añadió `server.port=${PORT:8080}` en `application.properties`, porque Render asigna el puerto mediante la variable de entorno `PORT`.

3.2 Docker Hub

Se creó la cuenta **davidarango6** y el repositorio público **lab2p2026**. Para que GitHub Actions pueda publicar imágenes sin usar la contraseña personal, se generó un token de acceso personal con permisos *Read & Write*.

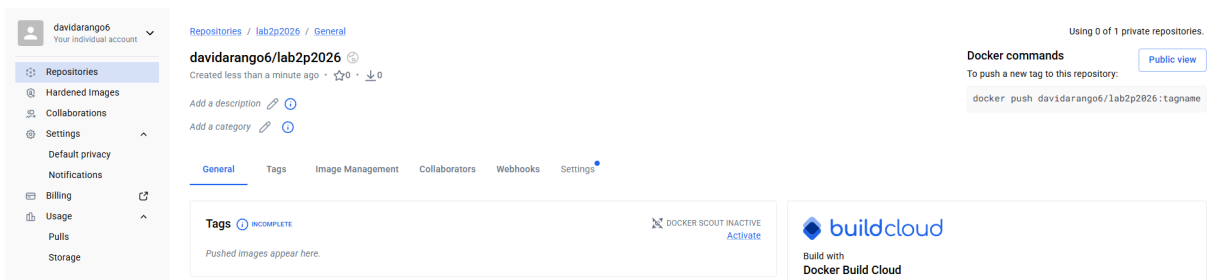


Figura 1. Repositorio davidarango6/lab2p2026 creado en Docker Hub.

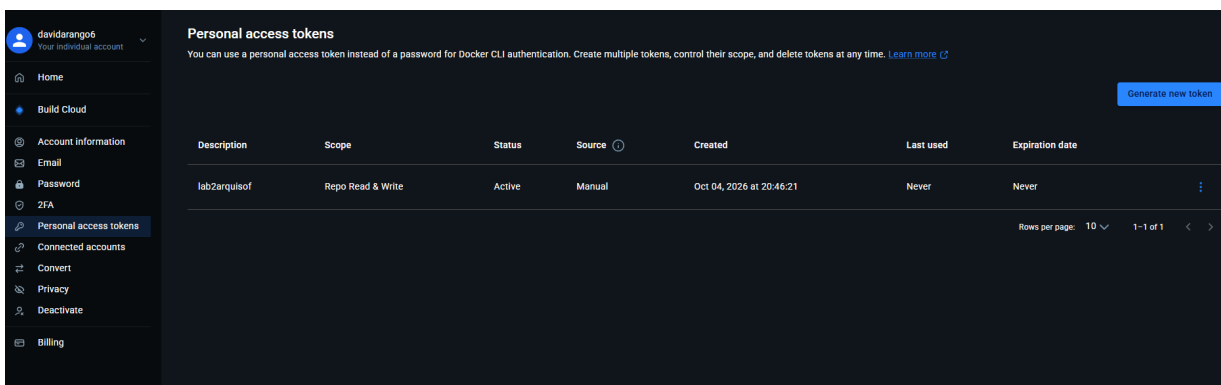


Figura 2. Token de acceso personal de Docker Hub (Read & Write).

3.3 Secrets de GitHub

En *Settings, Secrets and variables, Actions* se guardaron las credenciales como secretos cifrados, de modo que nunca aparecen en el código: `DOCKER_USERNAME`, `DOCKER_PASSWORD` (el token), `SONAR_TOKEN` y, más adelante, `RENDER_DEPLOY_HOOK_URL`.

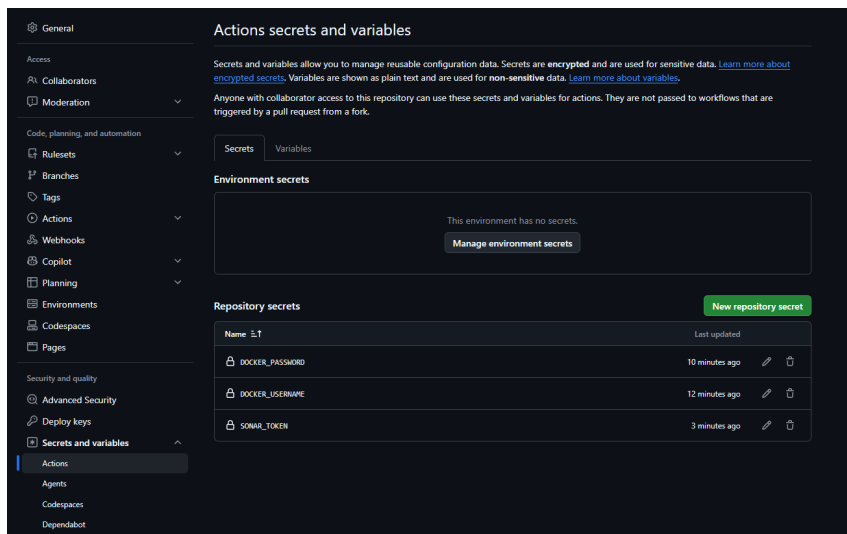


Figura 3. Primeros tres secretos del repositorio.

3.4 SonarCloud

Se importó el repositorio en SonarCloud (plan gratuito para repositorios públicos), se desactivó el *Automatic Analysis* para analizar solo desde GitHub Actions y se generó un token. Los datos del proyecto en `pom.xml` y en el workflow son:

```
sonar.projectKey      = DavidAraangoPineda_lab2p2026
sonar.organization    = davidaraangopineda
sonar.host.url        = https://sonarcloud.io
```

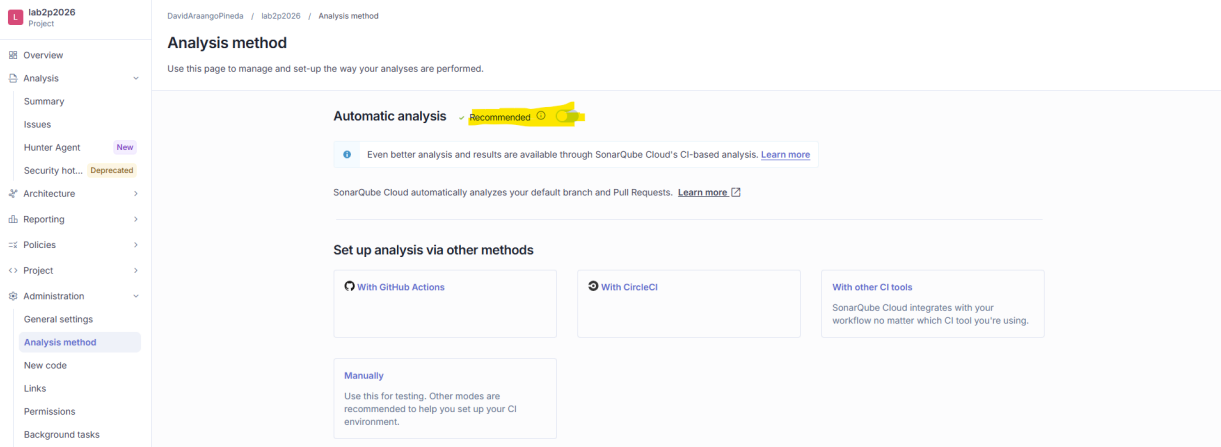


Figura 4. Automatic Analysis desactivado en SonarCloud.

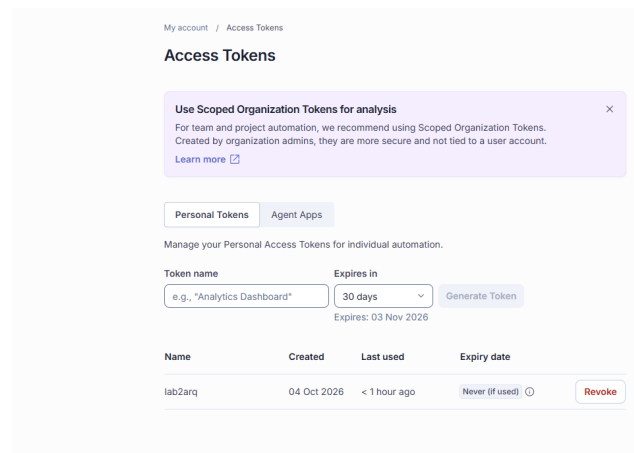


Figura 5. Token de SonarCloud para el pipeline.

3.5 Rama de trabajo y cambios en el workflow

Siguiendo *trunk based development* se creó la rama corta `feature/deploy-render` a partir de `main`. En ella se agregó el trabajo de despliegue a Render:

```
deploy:
  needs: build
  name: Deploy to Render
  if: github.ref == 'refs/heads/main' && github.event_name != 'pull_request'
  runs-on: ubuntu-latest
  steps:
    - name: Trigger Render deploy
      env:
        RENDER_DEPLOY_HOOK_URL: ${ secrets.RENDER_DEPLOY_HOOK_URL }
      run: curl --fail -X POST "$RENDER_DEPLOY_HOOK_URL"
```

La condición `if` hace que solo se despliegue lo que ya fue integrado en `main`; los pull requests y las ramas de trabajo ejecutan pruebas, análisis y construcción, pero no publican nada en producción.

3.6 Primera ejecución: error y corrección

El primer push de la rama falló en el trabajo de SonarCloud. El registro indicaba Organization key 'DavidAraangoPineda' does not exist: la clave de organización de SonarCloud es **davidaraangopineda** (minúsculas), mientras que 'DavidAraangoPineda' es solo el nombre visible. Se comprobó con la API pública de SonarCloud, se corrigió en `pom.xml` y `build.yml` y se volvió a subir.

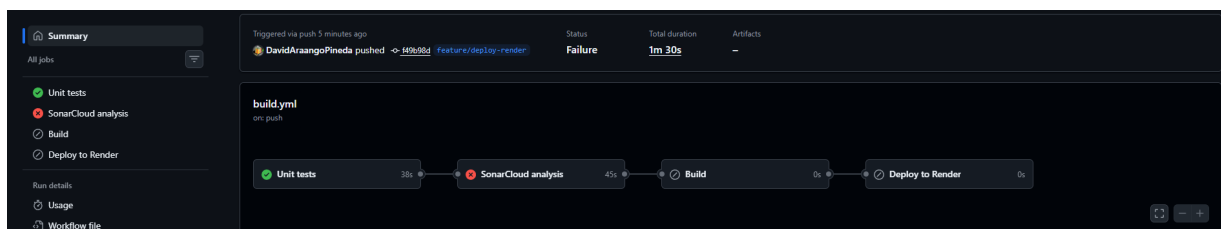


Figura 6. Ejecución fallida: el trabajo de SonarCloud falla por la clave de organización.

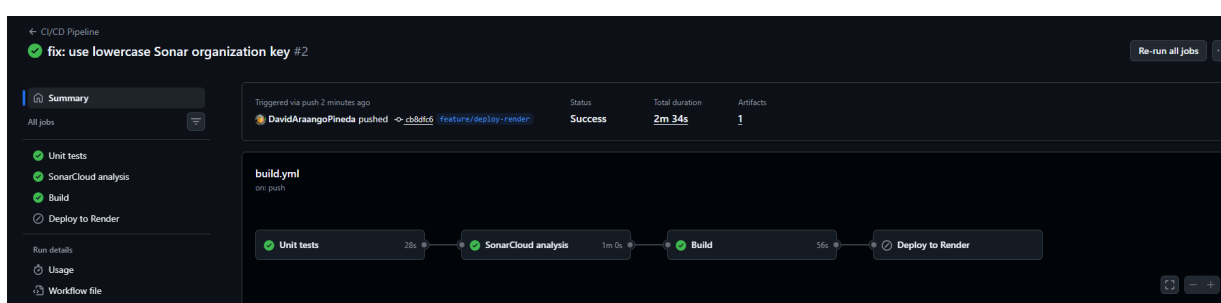


Figura 7. Tras la corrección, Unit tests, SonarCloud y Build pasan; Deploy se omite porque la rama no es main.

3.7 Imagen publicada en Docker Hub

El trabajo Build construyó la imagen con el Dockerfile multietapa (Maven para compilar y Temurin JRE 17 para ejecutar) y la publicó como `davidarango6/lab2p2026:latest`.

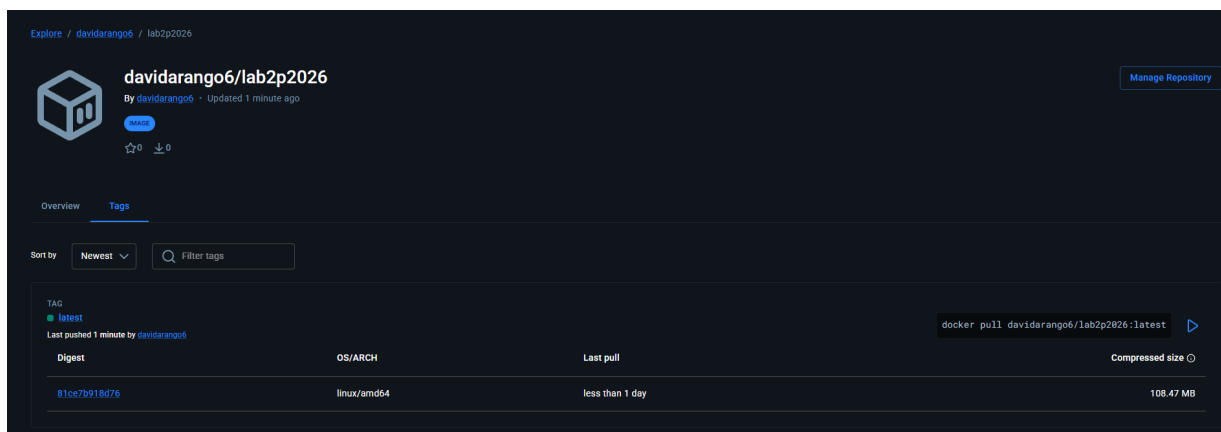


Figura 8. Imagen latest publicada en Docker Hub (linux/amd64).

3.8 Servicio en Render

En Render se creó un *Web Service* de tipo *Existing Image* con el plan gratuito, apuntando a `docker.io/davidarango6/lab2p2026:latest` (el repositorio es público, no requiere credenciales). De su configuración se copió el *Deploy Hook*, una URL secreta que se guardó en GitHub como `RENDER_DEPLOY_HOOK_URL`.

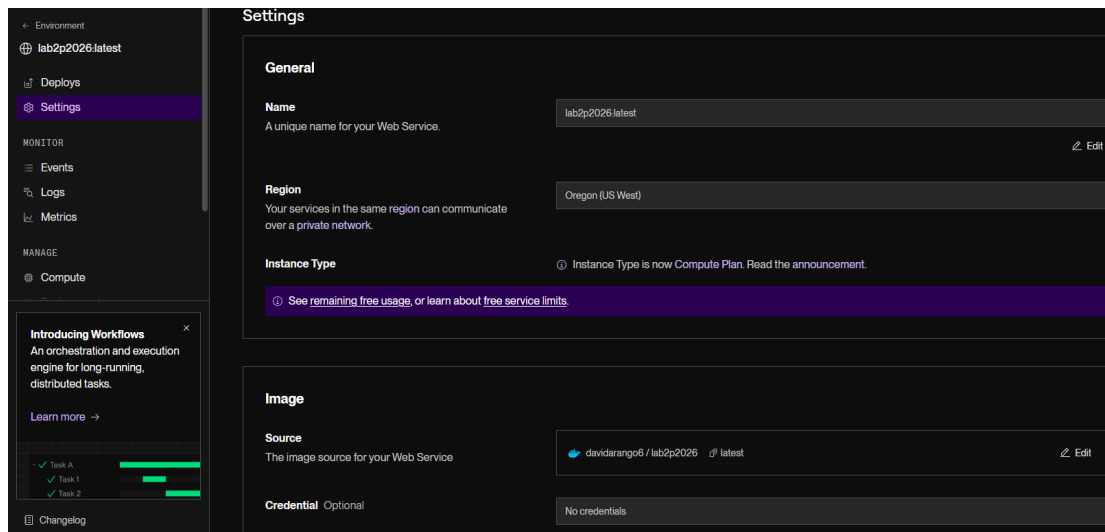


Figura 9. Configuración del servicio: imagen davidarango6/lab2p2026:latest sin credenciales.

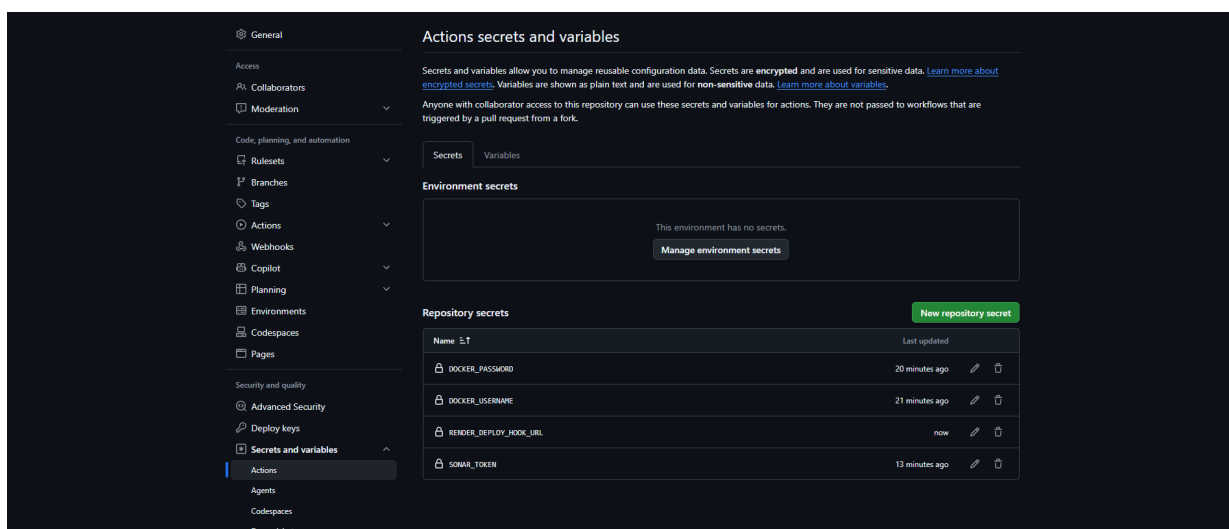


Figura 10. Los cuatro secretos del repositorio, incluido el deploy hook de Render.

3.9 Pull request y Quality Gate

Se abrió el pull request #1 de feature/deploy-render hacia main. El pipeline se ejecutó sobre el PR y SonarCloud comentó el resultado. La primera vez el *Quality Gate* falló por la condición **Security Rating on New Code (C)**.

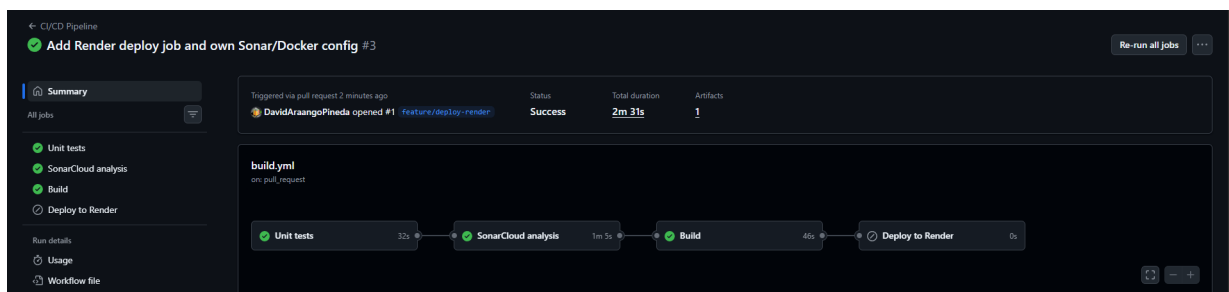


Figura 11. Pipeline del pull request en verde.

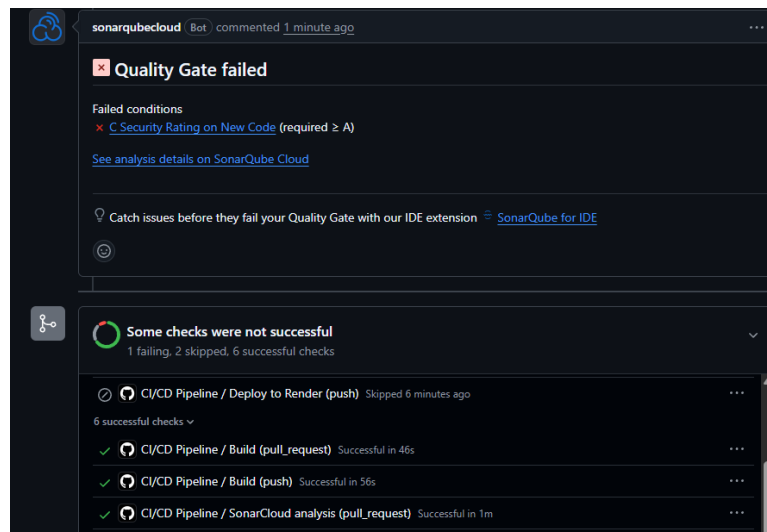


Figura 12. Quality Gate fallido: Security Rating C en el código nuevo.

Al consultar el detalle en SonarCloud apareció una única vulnerabilidad, regla githubactions:S7637: la acción de terceros mr-smithers-excellent/docker-build-push@v6 estaba referenciada por una etiqueta mutable. La buena práctica es fijarla al hash completo del commit, para que nadie pueda cambiar el código que ejecuta el pipeline sin que se note:

```
uses: mr-smithers-excellent/docker-build-push@05b44e1dd9e03f78e8689bd1a61c750f17907ffe # v6
```

Tras el cambio el Quality Gate pasó (0 incidencias nuevas, 0 hotspots de seguridad) y se fusionó el PR en main.

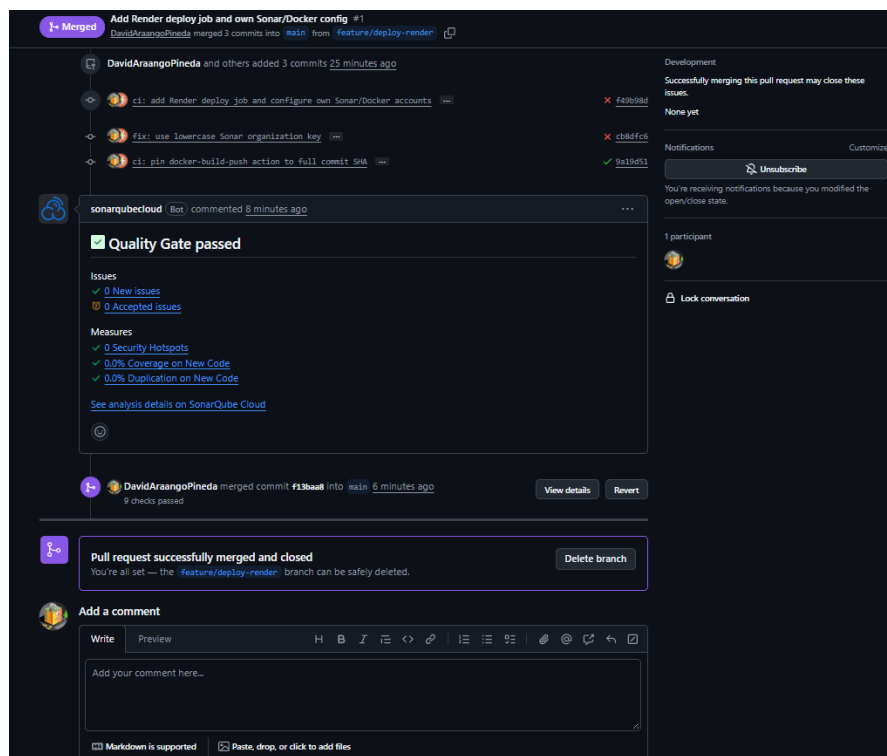


Figura 13. Pull request #1 fusionado con el Quality Gate en estado Passed.

3.10 Despliegue automático desde main

La fusión en main disparó el pipeline completo. Esta vez el trabajo **Deploy to Render** sí se ejecutó y llamó al deploy hook; Render descargó la nueva imagen y marcó el despliegue como

Live (disparador: *Deploy Hook*).

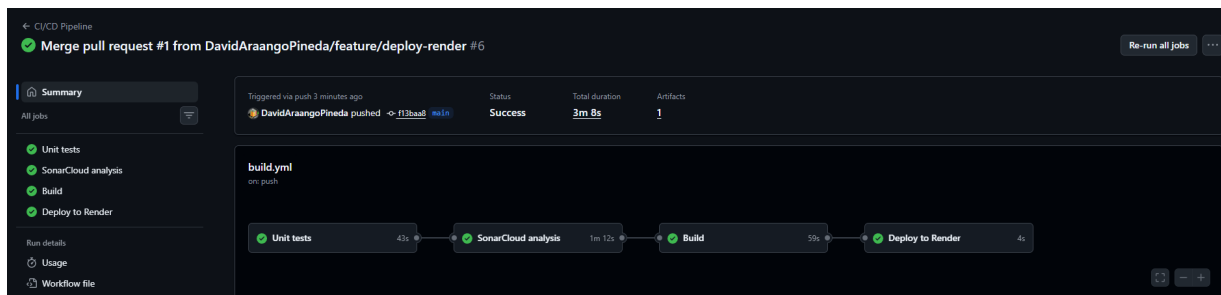


Figura 14. Pipeline en main: los cuatro trabajos en verde, incluido Deploy to Render.

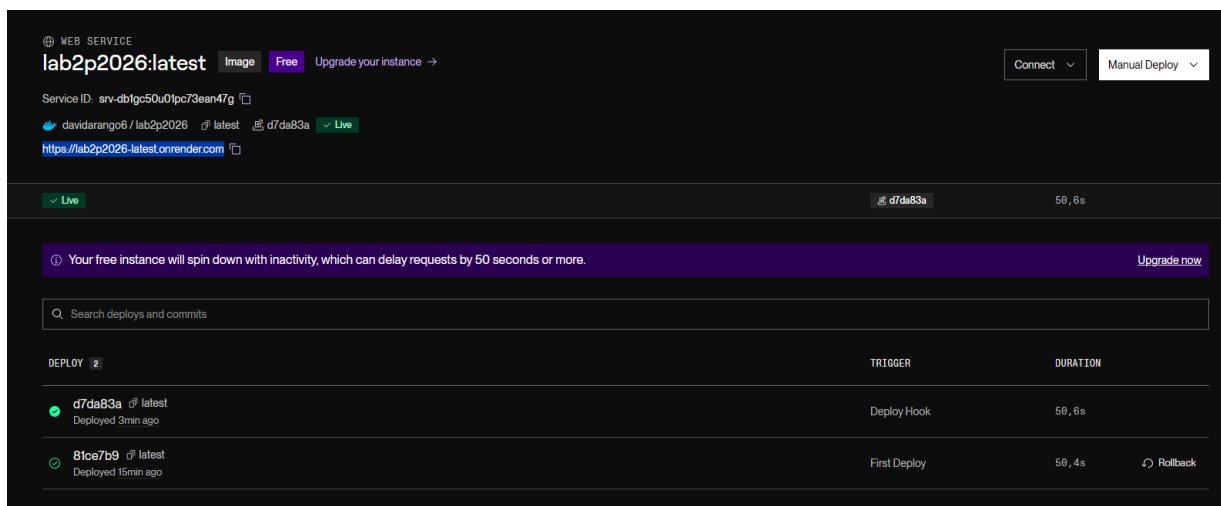


Figura 15. Render: despliegue activo (Live) disparado por el Deploy Hook.

3.11 Calidad del código en SonarCloud

El panel del proyecto muestra el Quality Gate en **Passed**, cobertura de **93,0 %** (medida con JaCoCo) y duplicación de **0,0 %**.

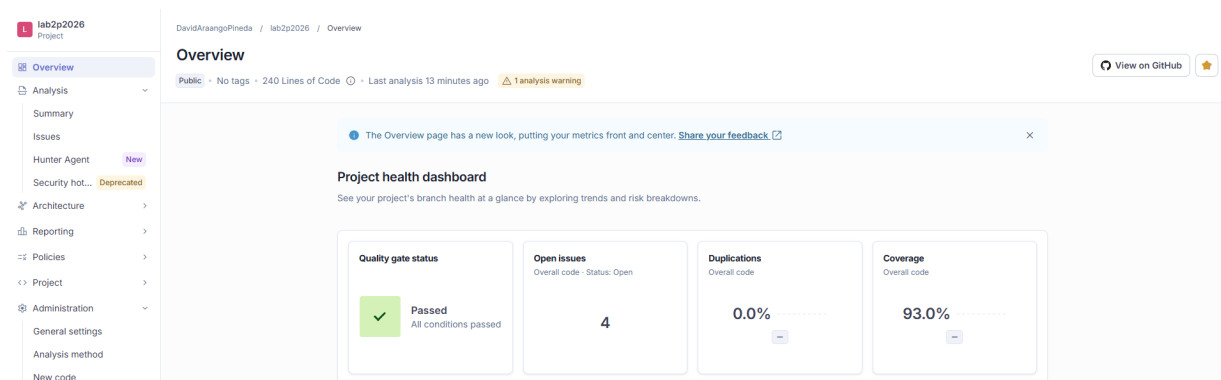


Figura 16. Panel de SonarCloud del proyecto.

4. Pruebas de ejecución del backend en la nube

Se probaron con Postman las cinco rutas de la API contra la URL pública <https://lab2p2026-latest.onrender.com>, todas con el método GET. Todas respondieron **200 OK**. Los tiempos más largos de algunas rutas se deben al plan gratuito de Render (poca CPU y memoria, y suspensión por inactividad).

Ruta	Respuesta esperada	Resultado
/	HEALTH CHECK OK!	200 OK

Ruta	Respuesta esperada	Resultado
/version	The actual version is 1.0.0	200 OK
/nations	Lista de 10 naciones (JSON)	200 OK
/currencies	Lista de 20 monedas (JSON)	200 OK
/aviation	Lista de 20 registros de aviación (JSON)	200 OK

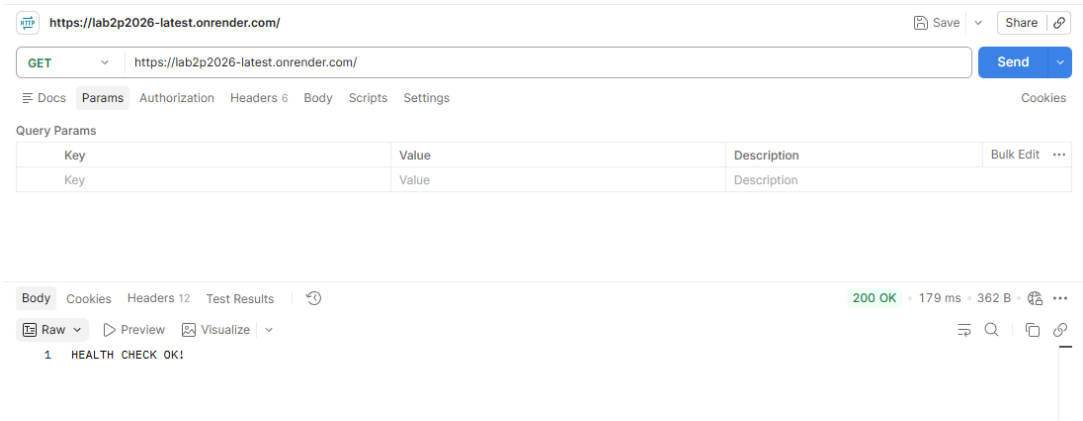


Figura 17. GET / (chequeo de estado).

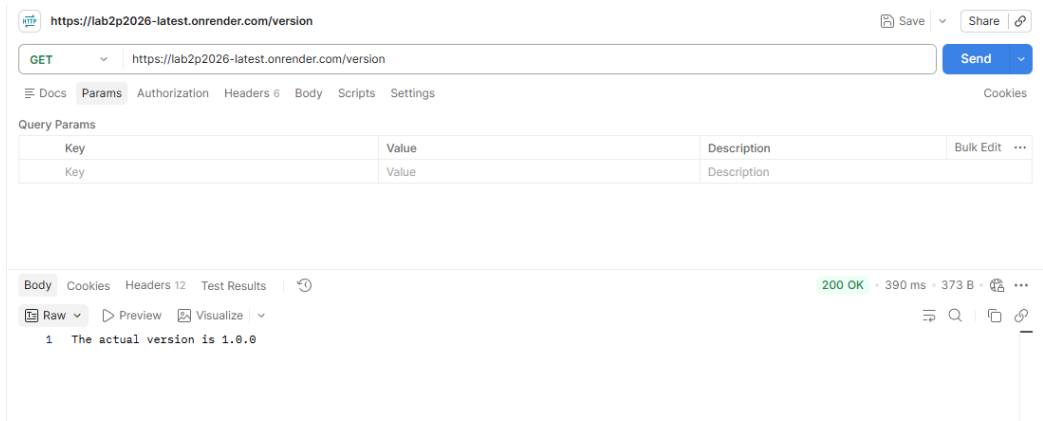


Figura 18. GET /version.

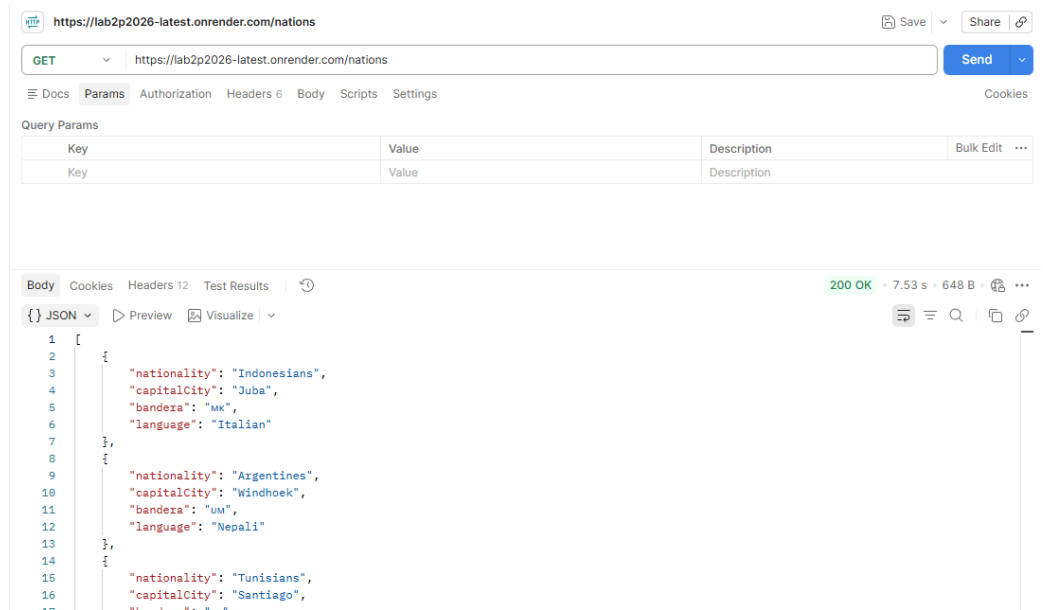


Figura 19. GET /nations.

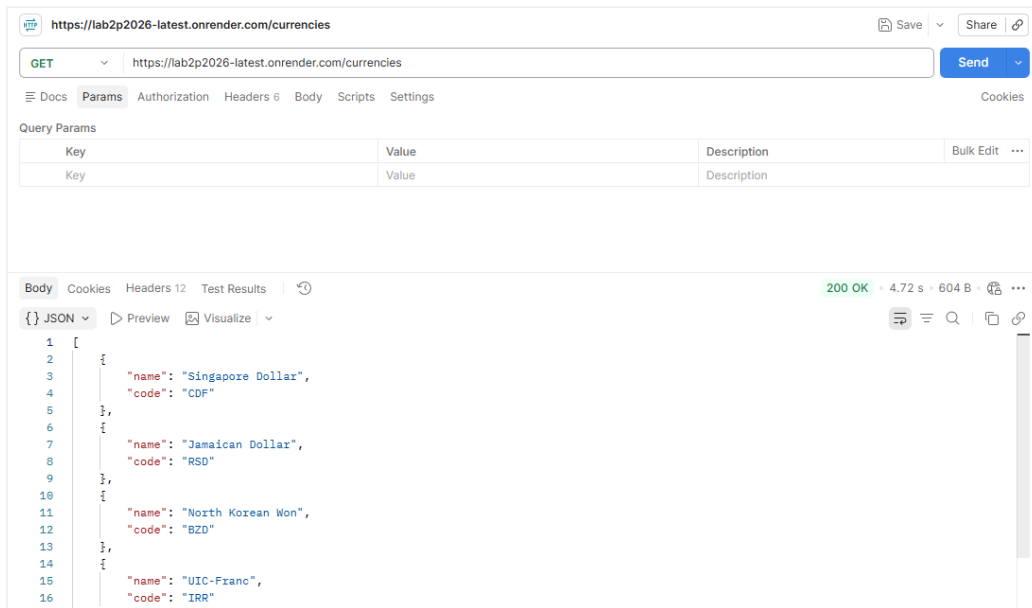


Figura 20. GET /currencies.

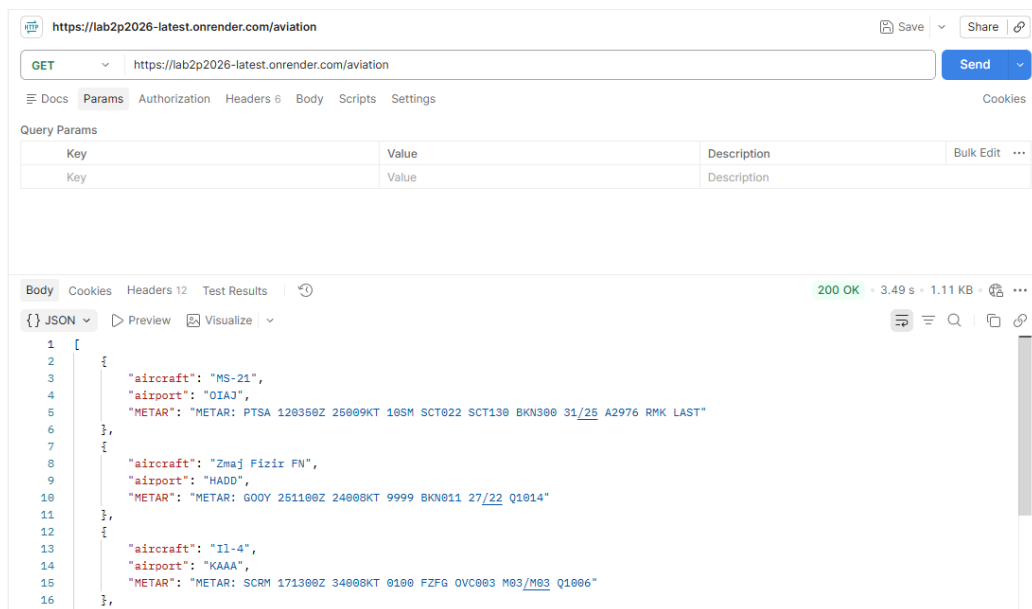


Figura 21. GET /aviation.

5. Problemas encontrados y soluciones

Problema	Causa	Solución
SonarCloud: 'Organization key does not exist'	La clave de organización es en minúsculas; se había usado el nombre visible.	Usar davidaraangopineda en pom.xml y build.yml.
Quality Gate fallido (Security Rating C)	Acción de terceros referenciada por etiqueta mutable (regla S7637).	Fijar la acción al hash completo del commit.
Postman devolvía 404 en rutas válidas	La URL copiada incluía un salto de línea al final (%0A en la ruta).	Borrar el salto de línea; las rutas responden 200.
502 y respuestas lentas ocasionales en Render	Plan gratuito con poca CPU/memoria y suspensión por inactividad.	Reintentar tras unos segundos; mejora posible: crear Faker una sola vez.

Problema	Causa	Solución
Badges del README apuntaban al proyecto del profesor	El README se heredó con las claves de diegobotia.	Actualizarlos a DavidAraangoPineda en la rama fix/informe-readme.

6. Segundo pull request (trunk based)

Para mostrar un segundo ciclo de integración, se creó la rama corta `fix/informe-readme`, que corrige los badges del README para que apunten al repositorio y al proyecto de SonarCloud propios y agrega este informe en `docs/Informe_Lab2_CICD.pdf`, enlazado desde el README. La rama se integra a `main` mediante un pull request; el pipeline se ejecuta sobre la rama y, tras la fusión, vuelve a desplegar en Render.

7. Conclusiones

Se logró un flujo completamente automatizado: un cambio fusionado en `main` se prueba, se analiza, se empaqueta, se publica como imagen y se despliega en la nube sin pasos manuales. Los secretos viven fuera del código, el despliegue solo ocurre desde la rama principal y el análisis de SonarCloud actuó como control de calidad real: detectó una mala práctica de seguridad en el propio pipeline que se corrigió antes de integrar. El uso de ramas cortas con pull requests dejó trazabilidad de cada cambio.